

I'll inspect the repository, trace its main execution paths, and check security, correctness, dependencies, tests, and deployment configuration. I'll run feasible checks in isolation and distinguish confirmed findings from unverified risks. No audit can honestly guarantee zero omissions; I'll document coverage and limitations. I won't modify the repository.

GitHub is not connected through Account / Connectors, so authenticated metadata is unavailable. I'll try a read-only public checkout to inspect and test the source; you don't need to connect GitHub if public access is sufficient.

The backend review has already identified serious issues: ownership is claimed using an unverified, publicly visible email string; viewer roles are not enforced on content and submission writes; and duplicate submission responses can disclose an existing record. I'm checking these against the tests and reviewing build reproducibility, query exposure, and frontend identity handling.

The frontend checks passed: 123 tests across 26 files, TypeScript, Biome, and the production build. But the backend test command exited successfully without running tests, reporting a stale WASM artifact. The dependency audit also reported 30 findings, including two critical-rated advisories; those ratings do not by themselves establish that the deployed app is exploitable.

I rebuilt the backend after adding the missing PocketIC toolchain in the temporary sandbox. All 15 existing backend tests passed.

Separate local probes nevertheless reproduced the ownership-claim flaw, viewer write access, disclosure through duplicate submissions, acceptance of invalid contact data, and storage of arbitrary analytics routes. They also confirmed a second, independent admin system in the inherited authorization package. This shows a security-coverage gap: the existing tests pass while these behaviors remain possible.

Ovanite repository audit

Verdict: I would block production approval until the ownership, authorization, privacy, and public-write issues below are fixed.

The project builds and its existing tests pass under a properly provisioned environment. However, **I reproduced several security failures against a freshly compiled canister in an isolated sandbox**. Passing tests currently do not establish that the application is secure.

Audited revision: [800406b6de77282a70cfa5976e20e911bc3ef3e4](#)

Audit/retrieval date: September 19, 2026

Repository inventory: 246 tracked files, including 18 Motoko files and 26 frontend test files.

I did not modify the remote repository or test against a live deployment. Toolchain adjustments and synthetic test data existed only in disposable sandboxes.

1. What actually passed—and what did not

Check	Result	Qualification
Frozen dependency installation	Passed	pnpm 10.17.1; lifecycle scripts disabled during installation
Frontend TypeScript check	Passed	Existing project configuration

Frontend tests	123 passed across 26 files	These do not establish backend security
Biome check	Passed	Checked 139 files
Frontend production build	Passed	Does not establish valid production runtime configuration
Motoko check	Passed	Included lint and stable compatibility checks
Backend build, unchanged configuration	Failed	Deployment check requires PocketIC, but the toolchain entry is missing
Backend build after sandbox-only toolchain addition	Passed	Added PocketIC 15.0.0
Repository's normal backend test command	Skipped tests and exited 0	Reported stale_backend_wasm
Backend tests through an explicit local PocketIC harness	15 passed	Used freshly compiled WASM
Additional security probes	Reproduced six unsafe behaviors	Detailed below
Resolved dependency audit	30 findings	2 critical, 13 high, 12 moderate, 3 low; application reachability varies
Limited current-tree secret-pattern scan	No matches	Not a comprehensive secret or history audit

The distinction between “**test command succeeded**” and “**tests executed successfully**” is particularly important here.

2. Confirmed security findings

Severity below is my assessment of the application issue, not a claimed formal CVSS calculation.

ID	Severity	Finding	Evidence level
S1	Critical, before ownership is claimed	Any non-anonymous caller knowing the public email string can become application owner	Reproduced locally
S2	High	Viewer accounts can perform content and submission writes	Viewer FAQ creation reproduced; other writes confirmed in source
S3	High	Duplicate submission responses disclose previously stored personal data	Reproduced locally
S4	High	Anonymous submission endpoints accept invalid data and lack application-level storage/abuse limits	Invalid contact accepted locally; limits absent in reviewed path

S5	High	Draft products and FAQs are exposed through public OQL entities	Confirmed in application and dependency source
S6	Medium	Analytics accepts arbitrary routes, permitting unbounded key growth and fabricated counts	Arbitrary route accepted locally
S7	Medium; higher if trusted by future integrations	Two independent admin systems disagree about who is an administrator	Reproduced locally

S1 – Ownership is based on a public string, not verified identity

Source: [admin-api.mo:43–54](#), [lib/admin.mo](#).

The ownership claim checks that the caller is non-anonymous and that the supplied text matches a hardcoded email. It does **not** verify control of that email or bind the claim to a pre-authorized principal.

Observed: An unrelated principal claimed ownership successfully on a fresh canister.

This is a first-claim takeover window, **not** proof that an already-claimed production deployment can be taken over through this method. The public `isOwnerClaimed()` method also reveals whether that window is open.

Fix: Bootstrap ownership through a trusted deployment/controller operation or a preconfigured principal. If email-based onboarding is required, verify a trusted identity assertion server-side and bind it to the caller. Merely hiding or changing the hardcoded email is not a security fix.

Ownership recovery also needs an explicit design: the current owner cannot be removed or demoted through the normal admin API.

S2 – Viewer permissions are descriptive, not enforced

Source: [types/admin.mo:7–16](#), [content-api.mo](#), [submissions-api.mo](#).

The declared contract says viewers are read-only. In practice, write guards only check membership in the `admins` map.

Observed: An account assigned `viewer` created an FAQ.

The same membership-only protection guards publishing, deletion, site-copy updates, submission-status changes, and submission deletion.

Fix: Separate read authorization from write authorization. Enforce owner/editor permissions in every backend mutation; disabling frontend buttons is insufficient.

S3 – Deduplication leaks another submitter’s record

Source: [lib/submissions.mo:27–98](#), [types/submissions.mo:42–49](#).

Duplicate responses return the entire existing `Submission`, including name, email, message, review status, and timestamps.

For waitlist entries with no note, matching requires only the email and an absent/empty note. The supplied name does not participate in matching.

Observed: A second synthetic request using a different name received the original subject’s name.

This creates both a membership-discovery mechanism and a personal-data disclosure. Contact duplicates require matching the message too, so their guessing prerequisites are stronger.

Fix: Return a minimal acknowledgement without the stored record. Prefer indistinguishable public responses where membership privacy matters. Changing `SubmitResult` requires regenerated bindings and frontend/test updates.

S4 – Public submissions have insufficient validation and abuse resistance

Source: [submissions-api.mo:24–38](#), [lib/submissions.mo](#).

Frontend validation is bypassable by calling the canister directly.

Observed: Empty name, malformed email, and empty contact message were stored successfully.

There are no application-level field-size limits, submission quotas, retention bounds, or abuse controls in this path. Platform message limits do not replace these controls.

Deduplication scans existing submissions and repeatedly normalizes their contents. A sequence of unique submissions therefore incurs approximately quadratic cumulative comparison work as the dataset grows. Listing additionally materializes and sorts the full matching collection without pagination.

Fix: Validate before storage; bound field sizes and retained records; add appropriate anonymous-abuse controls; maintain a normalized duplicate index; paginate administrative reads. Do not rely solely on IP-based controls for direct canister calls.

S5 – OQL bypasses the publication boundary

Source: [main.mo:100–143](#).

Dedicated public endpoints filter to published content. However, the OQL entities wrap the **entire** product and FAQ maps and declare them `.public_()`.

I inspected the installed OQL dependency: its map adapter iterates all values, and public authorization grants unrestricted row reads.

Consequently, “draft” is not a confidentiality boundary across the exported API surface.

Fix: Build public OQL entities from a published-only iterator, or restrict those entities. Filtering only the normal website endpoint does not resolve this.

Important distinction: Submission and admin OQL entities are marked controller-only. I did not find the same anonymous OQL exposure for those entities.

S6 – Analytics is neither route-bounded nor trustworthy

Source: [lib/analytics.mo](#), [analytics-api.mo](#).

Path normalization rejects certain malformed inputs and admin paths, but it does not restrict paths to actual public routes.

Observed: A nonexistent route created an analytics record.

A 128-character limit bounds each key, not the number of keys. Anonymous callers can also repeatedly increment counts. Additionally, raw input is processed before the final normalized-length check.

Fix: Enforce a finite route set or bounded aggregation model, reject oversized raw input early, and document these as untrusted event counts rather than verified visitors.

S7 — Parallel authorization systems create inconsistent privileges

Source: [main.mo:79–96](#) installed `caffeineai-authorization@1.0.1`.

The application maintains its own admin map while also including an authorization mixin with separate state. That package assigns its first non-anonymous initializing caller the package-level admin role.

Observed: A principal obtained `isCallerAdmin() === true` while `getCallerAdmin()` returned no application admin record.

This does **not** by itself bypass the custom content guards, which use the application map. It does make exported authorization signals inconsistent and creates a dangerous integration trap.

Fix: Choose one authoritative permission model. Remove unnecessary inherited role-management endpoints or explicitly reconcile their semantics and bootstrap policy.

3. Correctness, deployment, and assurance findings

ID	Priority	Finding	Recommended correction
R1	High when upgrading populated state	FAQ migration overwrites IDs 1–5 and resets the next ID to 6	Preserve existing rows and allocate collision-free IDs
R2	High assurance gap	Backend test infrastructure can skip execution and still return success	Require executed backend tests in release mode
R3	Medium	Missing PocketIC toolchain prevents the configured clean backend build	Pin the required toolchain and document provisioning
R4	Medium	Private frontend query keys are not identity-scoped	Scope keys by principal; remove private data during identity changes
R5	Medium, deployment-dependent	Checked-in runtime configuration contains "undefined" placeholders	Validate configuration and deployment rewriting before release
R6	Low/Medium performance	Production minification is disabled	Enable minification and measure actual delivered bundles

Migration data-loss risk

[20260918_130000.mo:105–165](#) unconditionally inserts FAQ IDs 1–5 into the existing map and sets `nextFaqId` to 6.

If the preceding version accumulated FAQs, those records can be overwritten. If ID 6 already exists, subsequent creation can overwrite that record too.

This is a **source-confirmed conditional upgrade defect**, not an observed production loss. Stable-type compatibility checks do not prove semantic data preservation.

Backend tests silently skip

[run-backend-lane.mjs](#) deliberately treats numerous infrastructure failures as successful exits.

Its freshness check compares modification times. A clean checkout can therefore trigger “stale” without establishing that committed WASM differs from the source.

Observed: The normal command exited 0 without tests. A separate harness executed all 15 tests successfully.

Release CI should distinguish optional local skipping from mandatory execution, and use reproducible build provenance rather than file timestamps alone.

Frontend identity-cache concern

[useAdmin.ts](#) and [useAdminSubmissions.ts](#) use shared keys across identities.

The infrastructure dependency **does invalidate and refetch queries when actors change**. That is useful, but it does not synchronously remove previously cached private data.

A stale-display window remains plausible during account changes or failed refetches. I did **not** reproduce a complete browser-level cross-account disclosure; treat this as a targeted verification requirement, not a confirmed exploit.

Deployment and performance

The checked-in `env.json` lacks usable deployment values. Sitemap and robots files also depend on a platform rewriting `__PUBLIC_ORIGIN__`. A generic static upload is therefore not equivalent to a correctly configured deployment.

The largest checked-in JavaScript chunk is **1,495,406 bytes uncompressed**, and Vite sets `minify: false`. This is an optimization opportunity—not a measured Core Web Vitals failure.

4. Dependency audit

The live audit reported **30 findings across 17 package names**. Counts below include multiple advisories or affected version branches; they are not 30 demonstrated application exploits.

Package	Resolved version observed	Findings	Highest reported severity
seroval	1.5.1	1	Critical
vitest	3.1.4	2	Critical
vite	5.4.21	3	High
lodash	4.17.23	2	High
lodash-es	4.17.23	2	High
js-cookie	2.2.1	1	High
sharp	0.34.5	2	High
postcss	8.5.8	4	High
nanoid	3.3.11	3	High
browserslist	4.28.1	2	High
esbuild	0.21.5	1	Moderate
fflate	0.8.2 and 0.6.10	2	Moderate

@vitest/mocker	3.1.4	1	Moderate
baseline-browser-mapping	2.10.12	1	Moderate
quill	2.0.3	1	Low
@babel/core	7.29.0	1	Low
postcss-selector-parser	6.1.2	1	Low

I cross-checked the critical advisories against their published notices.

Seroval: [GHSA-mv8w-475r-vwqw](#) concerns untrusted deserialization with plugins. Version 1.5.3 fixes that advisory. I did not establish a reachable server-side deserialization path in this application, so **“production RCE confirmed” would be inaccurate.**

Vitest: [GHSA-5xrq-8626-4rwp](#) depends on exposed UI/API or specified Windows UI/browser-mode conditions. The ordinary headless test command is not evidence of those conditions. A separate audit finding also affects the older mocker path.

Upgrade dependencies deliberately, remove unused packages where justified, regenerate the lockfile, and rerun compatibility tests. Do not blindly force major transitive upgrades solely to obtain a zero-count audit.

5. Why the existing tests missed these issues

The frontend suite has useful coverage of forms, routes, consent, metadata, and admin screens. The backend suite exercises actual canister behavior.

However, its security model is incomplete: it primarily distinguishes anonymous callers from an owner, while omitting important adversarial and role-boundary cases. It also tests duplicate responses as intended functionality without asserting that they preserve privacy.

Required regression test	Expected secure result
Unrelated principal attempts initial ownership claim	Denied
Viewer attempts every mutation	Denied
Editor attempts owner-only administration	Denied
Duplicate public submission	No prior personal record disclosed
Empty, malformed, and oversized submissions	Rejected before storage
Anonymous OQL query for draft content	Draft rows unavailable
Arbitrary analytics route	Rejected or mapped to bounded storage
Identity changes while private requests are in flight	No previous-user data displayed
Upgrade with populated FAQs and a high next ID	Existing data and ID monotonicity preserved
Mandatory backend lane lacks infrastructure	Release fails explicitly

Runtime configuration remains unresolved	Deployment validation fails
--	-----------------------------

6. Recommended remediation order

Stage	Work	Completion criterion
P0 – Security boundary	Fix owner bootstrap, enforce roles, remove duplicate-record disclosure, close draft OQL exposure	Negative authorization/privacy tests pass against the real canister
P1 – Availability and data integrity	Add validation, bounded storage, deduplication indexing, pagination, and migration preservation	Abuse-boundary and populated-upgrade tests pass
P1 – Release assurance	Pin toolchain, require executed backend tests, verify runtime configuration	Clean checkout produces a tested, deployable artifact
P2 – Dependency hygiene	Upgrade vulnerable packages and remove demonstrably unused dependencies	Remaining advisories have documented reachability and disposition
P2 – Browser and performance	Harden identity caching, enable minification, run browser/accessibility/performance tests	Measured results meet explicit acceptance criteria

7. Scope limits

This was a substantial source, dependency, build, and local-runtime audit—not a claim of mathematically exhaustive verification.

Area	Remaining limitation
Production deployment	No live canister configuration, controllers, cycle balance, gateway, or headers verified
Browser behavior	No full browser penetration, accessibility, visual-regression, or load-testing campaign
Upgrade execution	Migration risk reviewed in source; populated historical upgrade scenarios not executed
GitHub administration	Branch protection, private security alerts, and external CI settings unavailable
Secret scanning	Limited current-tree patterns only; no comprehensive historical secret certification
Third-party code	Security-relevant package paths inspected, not every transitive dependency line
Artifact provenance	No proof that the deployed or committed WASM is byte-for-byte reproducible from the audited source

Bottom line: The repository has a working implementation and a useful testing foundation, but its current permission and privacy boundaries are unsafe. The priority is not a broad rewrite: it is a focused correction of those boundaries, backed by real-canister negative tests and a release process that cannot silently skip them.